Task 5 – Read Input and Convert Types

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 3 | Bloom's level: K3 – Apply

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task05)

1. input() always returns a string

Whatever the clerk types, `input()` returns a **str**: typing 8.5 gives "8.5", not the number 8.5. Arithmetic on it would fail or, worse, give nonsense ("8.5" * 2 is "8.58.5"). The program must convert explicitly: `int(text)` for whole numbers (e.g. number of days) and `float(text)` or `Decimal(text)` for litres, fat and SNF.

2. What happens with "8,5" or "eight"

`float("8,5")` and `float("eight")` both stop the program with `ValueError: could not convert string to float`. The comma is a decimal separator in some regions, and the clerk may well type it, so the program must reject it with a clear message rather than crash or guess.

3. Validating before converting (string methods)

read_entry.py

```
def read_float(prompt, low, high):
    """Keeps asking until the clerk types a number between low and high."""
    while True:
        text = input(prompt).strip()
        if not text.replace(".", "", 1).isdigit():
            print("  Please type digits with an optional decimal point, e.g. 8.5")
            continue
        value = float(text)
        if not (low <= value <= high):
            print(f"  Value must be between {low} and {high}")
            continue
        return value

litres = read_float("Litres  : ", 0.1, 200)
fat = read_float("Fat %   : ", 2.0, 7.0)
snf = read_float("SNF %   : ", 6.0, 10.0)
print(f"Accepted: {litres:8.1f} L  fat {fat:4.1f}  snf {snf:4.1f}")
```

Run with the inputs: eight, 8,5, 10, 9.5, 4.2, 8.7

```
Litres  : eight
  Please type digits with an optional decimal point, e.g. 8.5
Litres  : 8,5
  Please type digits with an optional decimal point, e.g. 8.5
Litres  : 10
Fat %   : 9.5
  Value must be between 2.0 and 7.0
Fat %   : 4.2
SNF %   : 8.7
Accepted:     10.0 L  fat  4.2  snf  8.7
```

`text.replace(".", "", 1).isdigit()` removes at most one decimal point and then requires every remaining character to be a digit. Two checks are made separately: *is it a number?* and *is it in the valid range?*, so each error message is specific.

4. Formatted output with f-strings

width, alignment (<, >) and precision (.2f)

```
print(f"{'ID':<6}{'Litres':>8}{'Fat':>6}{'SNF':>6}{'Rate':>8}{'Amount':>10}")
print(f"{fid:<6}{l:8.1f}{f:6.1f}{s:6.1f}{r:8.2f}{a:10.2f}")
```

Aligned table

```
ID      Litres   Fat   SNF    Rate    Amount
F001      10.0   3.5   8.5   30.00    300.00
F002       8.5   4.2   8.7   33.40    283.90
F003      12.0   3.1   8.3   27.80    333.60
```

`{amount:10.2f}` means: a float in a field 10 characters wide with exactly 2 decimals; < left-aligns text and > right-aligns numbers so decimal points line up.

Alternative Concept and Complexity Analysis

Alternative: skip the pre-check and use `try: float(text) except ValueError:`.

entry_input.py (excerpt)

```
import timeit

def is_number(text):
    """String-method check: optional spaces, digits with at most one decimal point."""
    t = text.strip()
    return t.replace(".", "", 1).isdigit()

def to_float_checked(text):
    if is_number(text):
        return float(text)
    return None

def to_float_try(text):
    try:
        return float(text)
    except ValueError:
        return None
```

Measured behaviour and time (entry_input.py)

```
input        is_number + float    try/except float
'8.5'                      8.5                 8.5
'8,5'                     None                None
'eight'                   None                None
' 8.5 '                    8.5                 8.5
'-1'                      None                -1.0
'1e3'                     None              1000.0
''                        None                None
'nan'                     None                 nan
'inf'                     None                 inf
timing for '8.5'    check-first    301 ns   try/except    128 ns
timing for 'eight'  check-first    165 ns   try/except    813 ns
```

Input	Check first (strip + isdigit)	try / except float()
"8.5"	accepted (8.5)	accepted (8.5)
"8,5"	rejected – clearly because of the comma	rejected via ValueError
" 8.5 "	accepted after strip()	accepted: float() ignores surrounding spaces
"eight"	rejected	rejected via ValueError
"-1", "1e3"	rejected (not plain digits)	**accepted** as −1.0 and 1000.0 – valid floats but invalid litres, so a range check is still needed
"nan", "inf"	rejected	**accepted** as float values; only a range check (which is False for nan) stops them

Time: both are O(1) per entry for a short string (O(L) for length L). For a valid entry the pre-check costs about 300 ns against 130 ns for try/except; for an invalid entry the pre-check costs about 165 ns against 810 ns because raising an exception is expensive. Since entries are typed by a person (seconds each), the difference is irrelevant. **Clarity:** try/except is shorter and handles every format float() understands, but it also accepts "nan"/"inf"; the string check is stricter. The robust answer is to use try/except *and* a range check, or the string check followed by the range check, as in the program.

Evidence

Correct conversion, rejection of invalid input and neatly aligned output, shown above. Source: Task05/read_entry.py, entry_input.py.